Un'ipotesi che vorrei considerare prevede la definizione di una cartella services/ che definisce (come per langs/) i dettagli implementativi delle immagini e dei relativi containers per i servizi, così ad esempio in compose.services.yaml i valori nei campi "image" per i servizi "db" e "cache" possono specificare i nomi previsti per le corrispondenti immagini

Sì: è un'ipotesi molto buona e coerente con la tua architettura. Una cartella `services/` rende PostgreSQL, Redis, Mailpit ecc. **componenti gestiti da** `docker-dev`, con immagine, template, volumi, healthcheck e script di setup centralizzati; `compose.services.yaml` diventa allora un file generato che referenzia le immagini convenzionali, ad esempio `dev.postgres` e `dev.redis`. Il campo `image` in Compose identifica l'immagine usata dal servizio, mentre `build:` può costruire quella immagine se assente. [1] [2]

Layout proposto

```
docker-dev/
├── docker-dev.sh
├── apply-templates.sh
├── docker-run-extra-args.sh
├── langs/
│   └── Python/
│       ├── templates/
│       └── bin/
└── services/
    ├── postgres/
    │   ├── service.yaml
    │   ├── templates/
    │   │   ├── Dockerfile
    │   │   ├── entrypoint.postgres.sh
    │   │   └── compose.yaml.j2
    │   └── bin/
    │       ├── 10.setup-volumes.sh
    │       └── 20.build.sh
    ├── redis/
    │   ├── service.yaml
    │   ├── templates/
    │   └── bin/
    └── mailpit/
```

```
        ├── service.yaml
        ├── templates/
        └── bin/
```

Il parallelismo con `langs/` è forte:

Concetto	`langs/Python`	`services/postgres`
Identità	Linguaggio di sviluppo	Servizio di supporto
Immagine gestita	`dev.python`	`dev.postgres`
Template	Dockerfile, entrypoint, Compose	Dockerfile, entrypoint, Compose
Volumi	`venv.dev.python.foo`	`data.dev.postgres.foo`
Script	setup, build, run	setup, build, healthcheck/init
Selezione	`--target=Python/foo`	`--with=postgres`

Manifest del servizio

Usa un piccolo `service.yaml` come contratto macchina-leggibile:

```
# docker-dev/services/postgres/service.yaml
name: postgres
image: dev.postgres
default_tag: "16"
profiles:
  - db
volumes:
  - name: data
    mount: /var/lib/postgresql/data
    pattern: "data.dev.postgres.{project}"
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
  interval: 5s
  timeout: 3s
  retries: 10
ports:
  - name: postgres
    container: 5432
    default_host: 15432
environment:
  required:
    - POSTGRES_USER
    - POSTGRES_PASSWORD
    - POSTGRES_DB
```

Per Redis:

```
# docker-dev/services/redis/service.yaml
name: redis
image: dev.redis
```

```
default_tag: "7"
profiles:
  - cache
volumes: []
ports:
  - name: redis
    container: 6379
    default_host: 16379
```

`40.services.sh` legge questi manifest e genera il Compose; non deve conoscere i dettagli interni di PostgreSQL o Redis.

`compose.services.yaml` generato

Per:

```
docker-dev --target=Python/foo --with=postgres,redis
```

`40.services.sh` può generare:

```
services:
  db:
    image: dev.postgres:16
    profiles: ["db"]
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    volumes:
      - db-data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:15432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d app"]
      interval: 5s
      timeout: 3s
      retries: 10

  cache:
    image: dev.redis:7
    profiles: ["cache"]
    ports:
      - "127.0.0.1:16379:6379"

  dev:
    environment:
      DATABASE_URL: postgres://app:app@db:5432/app
      REDIS_URL: redis://cache:6379/0
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
```

```
volumes:
  db-data:
    name: data.dev.postgres.foo
```

Il nome logico del servizio può restare `db` e `cache` per stabilità dell'interfaccia, mentre il **tipo di servizio** è `postgres` e `redis`. Questa separazione è utile: un progetto può sostituire `db` con MySQL senza cambiare il modo in cui lo richiama.

Immagini gestite

Seguirei la tua convenzione:

```
dev.python
dev.rust
dev.java

dev.postgres
dev.redis
dev.mailpit
dev.minio
```

Con tag espliciti:

```
dev.postgres:16
dev.redis:7
dev.mailpit:latest
```

Due strategie possibili:

Strategia	Esempio	Quando usarla
Immagine upstream con tag bloccato	`postgres:16.4`	Servizio standard, nessuna personalizzazione
Immagine `docker-dev` costruita	`dev.postgres:16`	Serve entrypoint, estensioni, locale, utente, healthcheck o configurazione comune

Se non devi personalizzare PostgreSQL, il manifest può dichiarare direttamente:

```
image: postgres:16.4
build: null
```

Se invece vuoi un'immagine gestita, il template genera:

```
FROM postgres:16.4
COPY entrypoint.postgres.sh /usr/local/bin/
RUN chmod +x /usr/local/bin/entrypoint.postgres.sh
```

E Compose può avere sia `image` sia `build`: Compose usa l'immagine esistente secondo la `pull_policy`; se non la trova, può costruirla dal sorgente indicato.[1]

Build delle immagini servizio

Aggiungi un comando esplicito:

```
docker-dev --build-service=postgres
docker-dev --build-service=postgres,redis
docker-dev --target=Python/foo --with=postgres,redis --build-missing
```

Il flusso consigliato:

```
1. 40.services.sh risolve i servizi richiesti
2. Per ogni servizio:
   - legge services/<name>/service.yaml
   - calcola image name/tag
   - se l'immagine non esiste e build è previsto:
       esegue services/<name>/bin/20.build.sh
3. Genera compose.services.yaml
4. Esegue docker compose
```

Eviterei il build implicito silenzioso come comportamento predefinito: meglio un messaggio chiaro tipo:

```
Image dev.postgres:16 not found.
Run: docker-dev --build-service=postgres
```

oppure consentire `--build-missing` come opt-in.

Volumi e naming

Mantieni la stessa disciplina dei progetti:

```
data.dev.postgres.foo
data.dev.redis.foo
uploads.dev.minio.foo
```

Ma attenzione: il nome del volume dovrebbe includere il **tipo di servizio**, non solo il nome logico:

```
data.dev.postgres.foo       # corretto
data.dev.db.foo             # ambiguo se db diventa MySQL
```

Se un progetto passa da PostgreSQL a MySQL, i volumi restano distinti:

```
data.dev.postgres.foo
data.dev.mysql.foo
```

Per i volumi dei servizi, puoi lasciare che Compose li dichiari e gestisca:

```
volumes:
  db-data:
    name: data.dev.postgres.foo
```

Se invece il servizio richiede inizializzazione di permessi o ownership, dichiaralo `external: true` e fallo creare da `services/postgres/bin/10.setup-volumes.sh`, esattamente come fai per `venv.dev.python.foo`.

Dipendenze dal servizio `dev`

Il file generato dovrebbe aggiungere al servizio `dev` solo ciò che dipende dalla composizione richiesta:

- variabili d'ambiente di connessione;
- `depends_on`;
- eventuali mount condivisi;
- alias di rete, se necessario.

Non dovrebbe modificare `entrypoint`, `command`, `user` o `working_dir` del servizio di sviluppo: questi appartengono al template del linguaggio. In un merge Compose, alcune chiavi come `command`, `entrypoint` e `healthcheck.test` vengono sostituite dall'ultimo file, quindi è meglio mantenere netta la responsabilità dei file. [3] [4]

Interfaccia Ddev

```
# Servizi con configurazione predefinita
docker-dev --target=Python/foo --with=db,cache

# Alias/logical service names
docker-dev --target=Python/foo --with=postgres,redis

# Configurazione esplicita
docker-dev --target=Python/foo \
  --with=postgres:tag=16,host_port=15432 \
  --with=redis:tag=7

# Build delle immagini servizio
docker-dev --build-service=postgres,redis

# Ispezione del Compose merged
docker-dev --target=Python/foo --with=postgres --config
```

Internamente puoi distinguere:

```
--with=db       -> logical alias -> service type postgres
--with=postgres -> service type postgres
```

Il primo è più stabile per gli utenti; il secondo è più esplicito e utile per progetti che usano più database.

Vantaggi di questo modello

- **Riuso reale**: `services/postgres` è identico per tutti i progetti Python, Rust o Java.
- **Interfaccia uniforme**: `--with=postgres` funziona allo stesso modo per ogni linguaggio.
- **Configurazione auditabile**: `service.yaml` dichiara immagine, volumi, porte, healthcheck e variabili richieste.
- **Nomi deterministici**: `dev.postgres:16`, `data.dev.postgres.foo`, `dev.python.foo` sono prevedibili.
- **Estensibilità**: aggiungere MinIO, Mailpit o un broker diventa creare una cartella in `services/`, non modificare `docker-dev.sh`.
- **Separazione netta**: `langs/` definisce l'ambiente di sviluppo; `services/` definisce le dipendenze esterne del progetto.

L'unico punto da curare è il contratto di generazione: `40.services.sh` dovrebbe essere un **resolver + generatore**, non contenere logica specifica di PostgreSQL o Redis. Tutta la conoscenza del singolo servizio resta in `services/<name>/`. Compose fornisce poi il meccanismo per combinare questi frammenti in un unico modello di applicazione. [5] [1]

⁂

1. https://docs.docker.com/reference/compose-file/build/
2. https://apxml.com/courses/docker-for-ml-projects/chapter-6-docker-compose-ml-workflows/defining-compose-services
3. https://github.com/Krishnamurtyp/docker-compose-spec/blob/master/13-merge.md
4. https://github.com/compose-spec/compose-spec/blob/main/13-merge.md
5. https://oneuptime.com/blog/post/2026-01-16-docker-compose-extends/view
6. https://oneuptime.com/blog/post/2026-01-25-docker-compose-extends/view
7. https://www.reddit.com/r/docker/comments/v5zwyk/override_parts_of_dockercomposeyml_and_reuse/
8. https://stackoverflow.com/questions/58673249/docker-compose-replacement-for-the-extends-keyword
9. https://docs.docker.com/compose/how-tos/multiple-compose-files/extends/
10. https://docs.docker.com/reference/compose-file/services/
11. https://docs.docker.com/reference/compose-file/
12. https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/
13. https://reintech.io/blog/customize-docker-compose-extension-fields-overrides
14. https://lours.me/posts/compose-tip-037-include-extends-overrides/
15. https://engineering.salesforce.com/containers-part-2-what-is-docker-compose-50a3a5685cc4/

16. https://github.com/YelpArchive/docker-compose/blob/master/docs/extends.md

17. https://www.geeksforgeeks.org/devops/docker-compose/